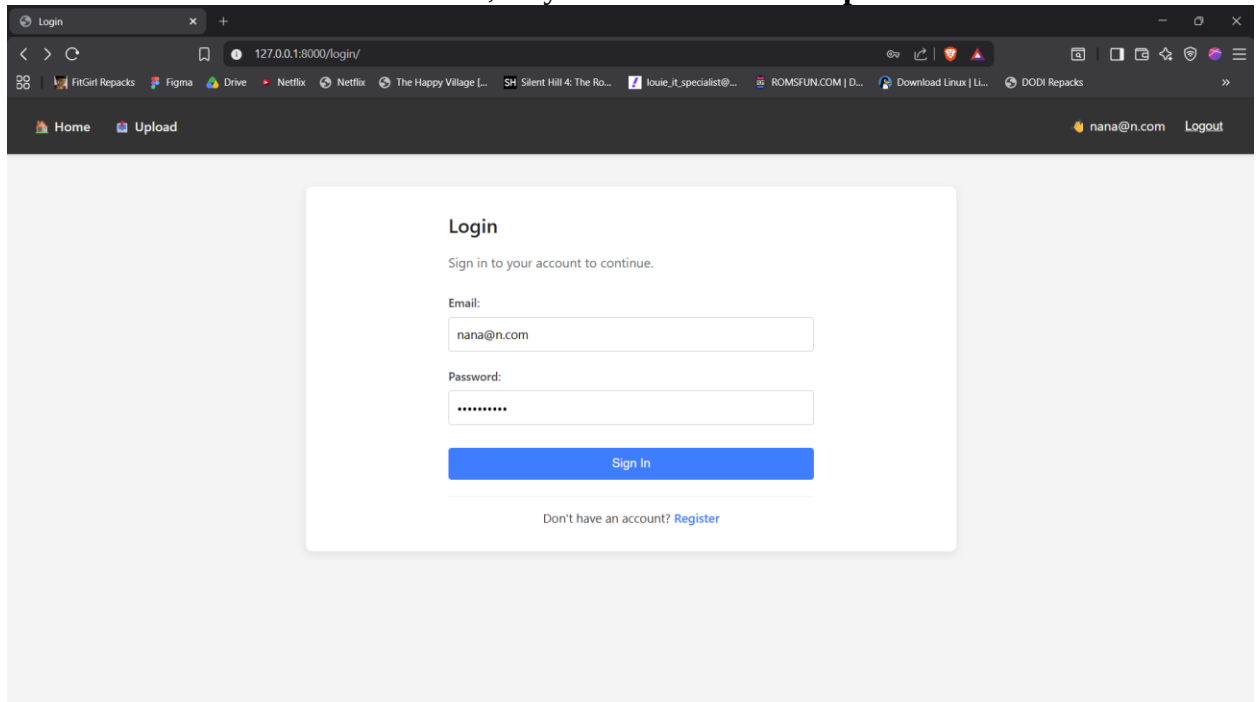


Project Design by Louie Logan

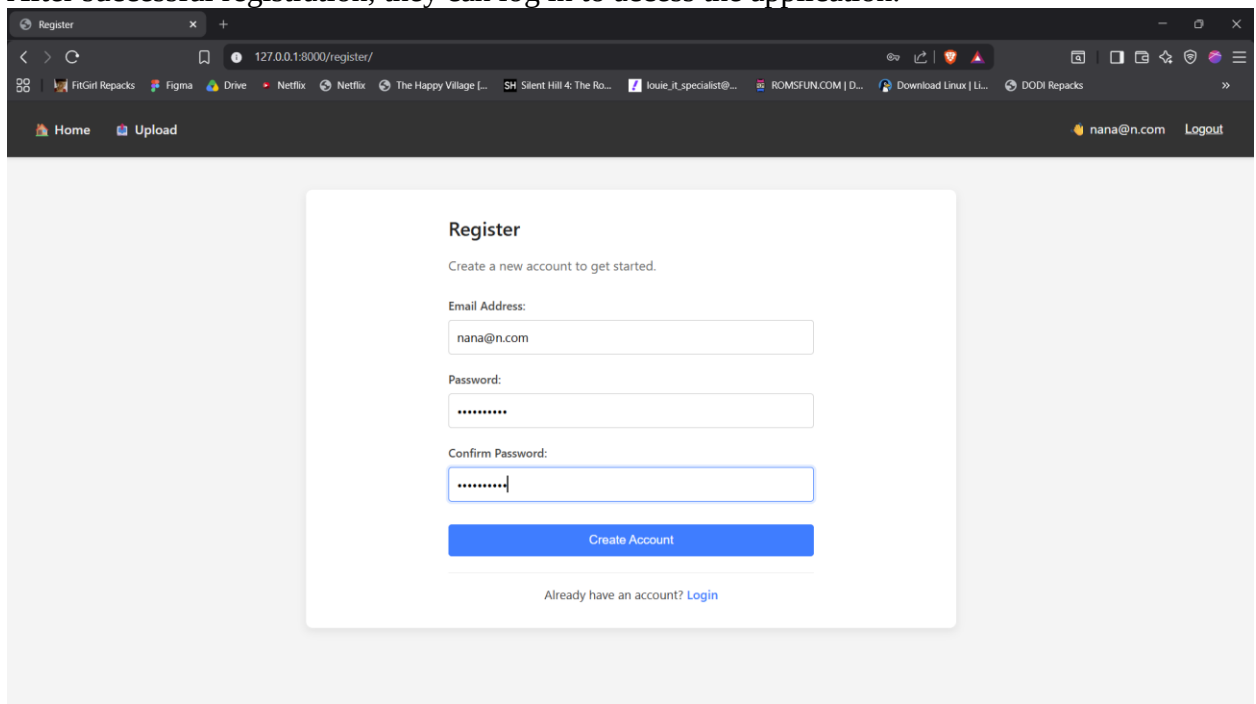
This document presents the design and implementation of the **Photo Classification Platform**, developed as part of a technical assignment for **WorkNomads**. The application is built using Django and follows a simple microservices architecture to classify uploaded photos as safe or unsafe. This report describes the system architecture, database design, security implementation, Docker-based microservices, and the proposed deployment and scaling strategy. It also includes screenshots, setup instructions, and a high-level system diagram.

Screenshots

- a. **Login Screen** : If the user logs in with an administrator account, they will be redirected to the **Admin Dashboard**. Otherwise, they will be taken to the **Update Screen**.

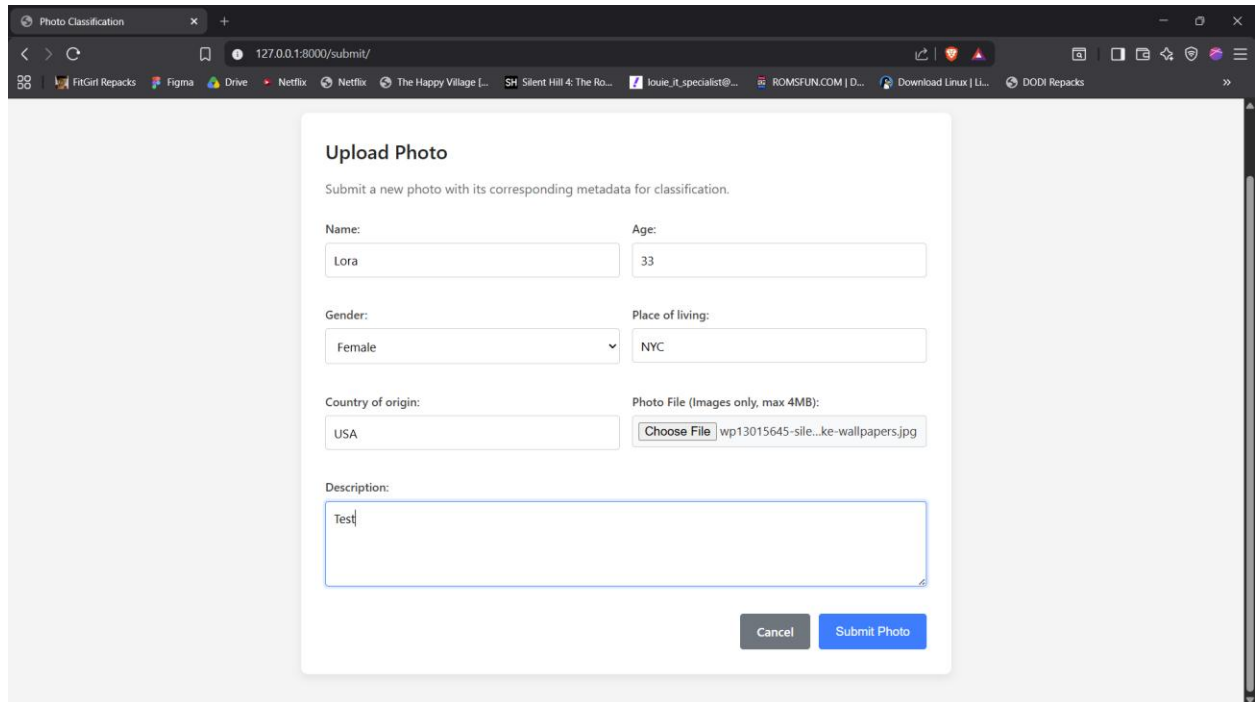


- b. **Register Screen** : New users can create an account by completing the registration form. After successful registration, they can log in to access the application.



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/register/". The browser's tab is labeled "Register". The page features a dark navigation bar at the top with "Home" and "Upload" links on the left, and a user profile "nana@n.com" with a "Logout" link on the right. The main content area is a light gray background with a white registration form centered. The form is titled "Register" and includes the instruction "Create a new account to get started." It contains three input fields: "Email Address:" with the value "nana@n.com", "Password:" with masked characters "*****", and "Confirm Password:" with masked characters "*****". A blue "Create Account" button is positioned below the fields. At the bottom of the form, there is a link: "Already have an account? [Login](#)".

- c. **Update Photo Screen** : This screen is available for regular users. Users can upload or update their photo and add their characteristics.



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/submit/". The browser's tab is titled "Photo Classification". The main content area features a white modal form titled "Upload Photo". Below the title, a subtitle reads "Submit a new photo with its corresponding metadata for classification." The form contains several input fields: "Name:" with the value "Lora", "Age:" with the value "33", "Gender:" with a dropdown menu showing "Female", "Place of living:" with the value "NYC", "Country of origin:" with the value "USA", and "Photo File (Images only, max 4MB):" with a "Choose File" button and the filename "wp13015645-sile...ke-wallpapers.jpg". A "Description:" text area contains the word "Test". At the bottom right of the form are two buttons: "Cancel" and "Submit Photo".

Upload Photo

Submit a new photo with its corresponding metadata for classification.

Name: Lora Age: 33

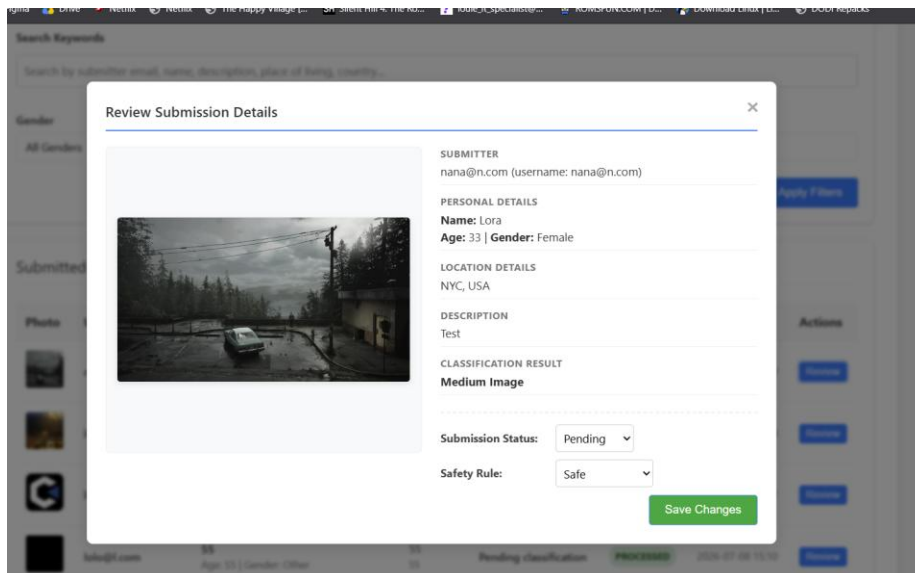
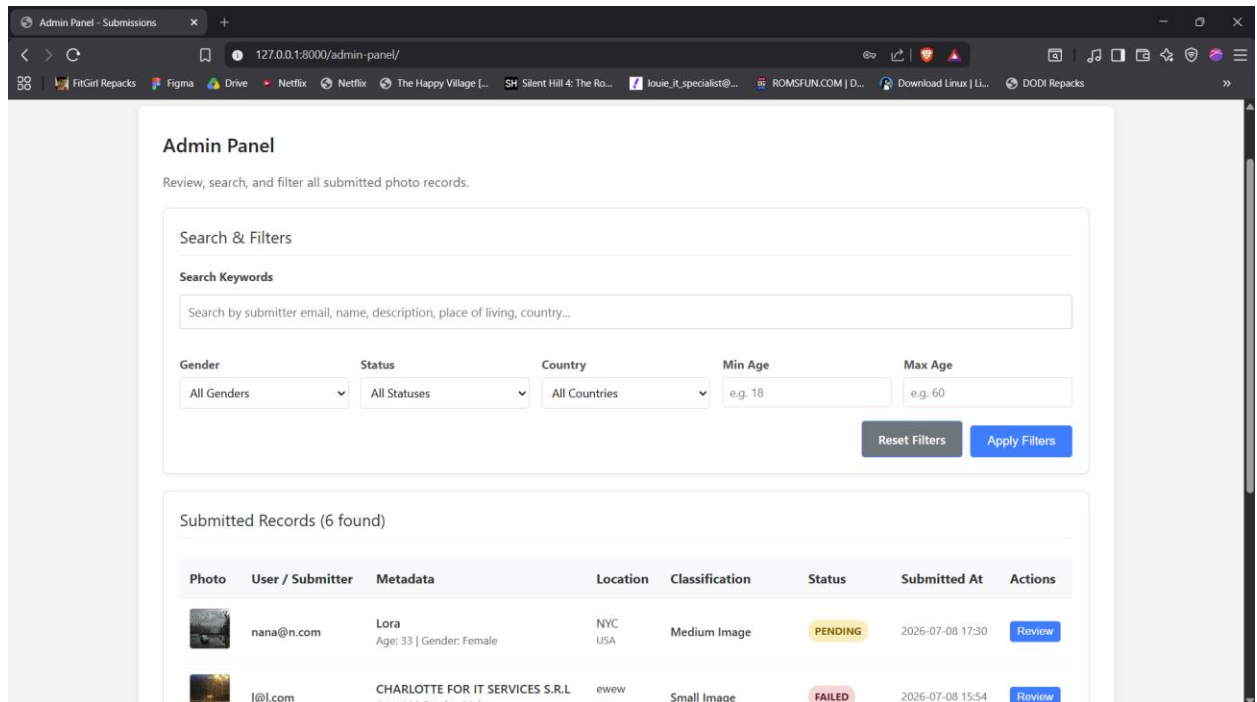
Gender: Female Place of living: NYC

Country of origin: USA Photo File (Images only, max 4MB): Choose File wp13015645-sile...ke-wallpapers.jpg

Description: Test

Cancel Submit Photo

- d. **Admin Panel:** This screen is accessible only to administrators. Admins can view all submitted users and their current statuses. They can filter users by gender, status, country, and age. Administrators can also update their status and review each submission through a popup window.



1. Safety Rule

I implemented a simple safety rule that prevents users from uploading images larger than **20 MB**. This helps prevent very large uploads from affecting the application's performance and storage

2. Database

I chose **SQLite** because it is lightweight, easy to configure, and suitable for the size and requirements of this project. It allows quick development without the overhead of a database server

3. Database Schema

- **Schema Design:** The database contains separate **User**, **Photo**, and **Submission** tables connected using **ForeignKey** and **OneToOne** relationships.
- **Migrations:** Django migrations were used to manage database changes during development
- **Indexing:** Indexes were added on **age**, **gender**, **country_of_origin**, **place_of_living**, and **created_at** to improve filtering and search performance

4. Security

- User authentication is handled by Django's authentication system.
- Only authenticated users can submit photos.
- The admin panel is accessible only to administrator accounts
- Input validation is used to reduce invalid data submissions
- A maximum upload size of **20 MB** is enforced

5. Docker and Microservices

The project contains two lightweight microservices:

- **Safe Rule Service** automatically sets `safety_rule = "safe"` for submissions with `status = "processed"`.
- **Unsafe Rule Service** automatically sets `safety_rule = "unsafe"` for submissions with `status = "failed"`.

Each service runs inside its own Docker container based on `python:3.12-slim`. The containers share the same `db.sqlite3` database through a Docker volume. Django's ORM is used with a minimal configuration, keeping each service lightweight. Docker Compose starts all services with a single command.

6. CI/CD

A simple CI/CD workflow is proposed for this project.

- Run linting to check code quality.
- Run automated tests.
- Build Docker images for the application and microservices.
- Push the images to a Docker container registry.
- Deploy the images to a Kubernetes cluster.

The CI/CD pipeline is documented but not fully implemented, as allowed by the assignment.

7. Kubernetes Strategy

Each service can run as its own Kubernetes Deployment with a corresponding Service. Environment variables and configuration can be managed using ConfigMaps and Secrets. This makes it easy to update or scale each service independently.

8. Scaling, Secrets, and Observability

- **Scaling:** Each microservice can be scaled independently by increasing the number of replicas.
- **Secrets:** Sensitive information such as secret keys and database credentials should be stored using Kubernetes Secrets instead of hardcoding them.
- **Observability:** Application logs can be collected using Docker or Kubernetes logging, and monitoring tools such as Prometheus and Grafana can be added in a production environment.

Setup Microservices

> Run all commands from the **project root**.

Build

bash

```
docker build -t safe-rule-service -f microservices/safe_rule_service/Dockerfile .
```

```
docker build -t unsafe-rule-service -f microservices/unsafe_rule_service/Dockerfile .
```

Run

powershell

```
docker run --rm -v "${PWD}/db.sqlite3:/app/db.sqlite3" safe-rule-service
```

```
docker run --rm -v "${PWD}/db.sqlite3:/app/db.sqlite3" unsafe-rule-service
```

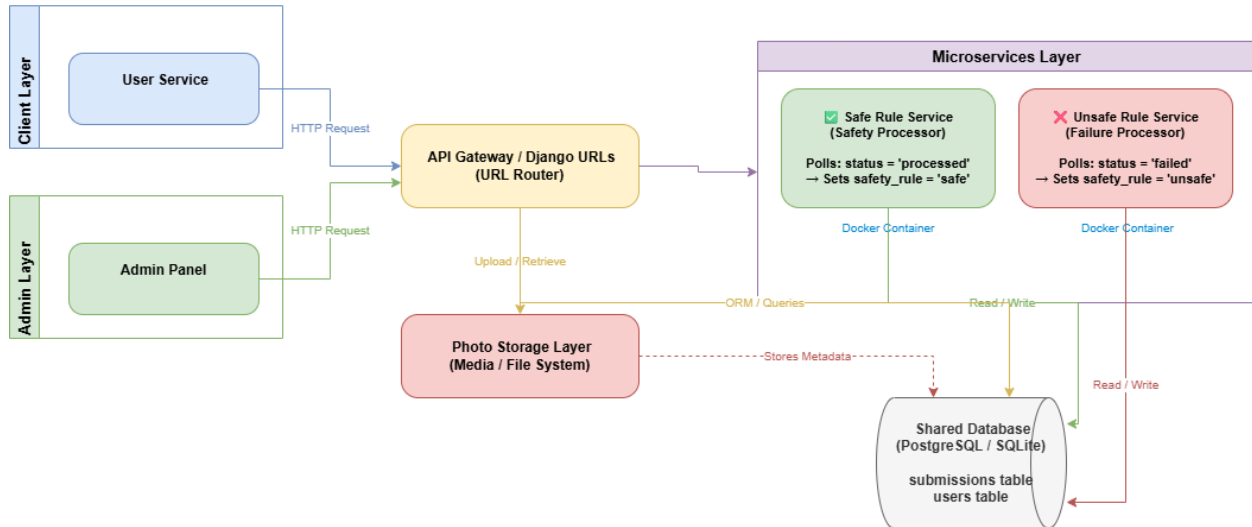
Or both at once

bash

```
docker compose -f microservices/docker-compose.yml up --build
```

Diagram

Photo Classification Platform Architectur



The **Photo Classification Platform** is a Django-based application that allows users to upload photos, which are then automatically classified as **safe** or **unsafe** by two independent microservices.

The **User Service** and **Admin Panel** communicate with the Django backend through HTTP requests routed by a central **Django URL Router (API Gateway)**. The backend manages file uploads to the **Photo Storage Layer** and interacts with the shared database using the **Django ORM**.

Two background microservices handle the classification process. The **Safe Rule Microservice** continuously polls the database for submissions with a `processed` status and updates them to `safe`, while the **Unsafe Rule Microservice** polls for submissions with a `failed` status and updates them to `unsafe`. Both services run independently in isolated **Docker containers**.

All components share a single **PostgreSQL or SQLite database**, which serves as the central source of truth for user accounts, photo submissions, and classification metadata.

User-facing operations use **synchronous communication** (HTTP requests and ORM database access), while the classification services use **asynchronous polling**, allowing the system to remain modular, scalable, and loosely coupled.